

Complete Solutions Manual

*Functional Programming Foundations
& Compiler Construction Laboratories*

October 03, 2026

Preface

This document serves as the exhaustive, rigorous **Solutions Manual** to accompany the course monograph *Principles of Functional Programming & Compiler Construction*. It collects complete derivations, proofs, Haskell implementations, Alex/Happy grammar definitions, and virtual stack machine symbolic assembly sequences for:

1. All embedded lecture and chapter exercises across functional programming and compiler topics.
2. The foundational problem sets (including well-parenthesized expression grammar variations).
3. The reference architectures and implementation strategies for the 24-milestone Scientific Calculator project.
4. The reference architectures and implementation strategies for the 42-milestone Pascal-to-Stack-Bytecode Compiler project.

Contents

Preface	i
I Part I: Solutions for Functional Programming Foundations	1
1 Chapter 1: Paradigms of Computation & Lambda Calculus	1
2 Chapter 2: Algebraic Data Types & Recursive Data Modeling	2
3 Chapter 3: Evaluation Strategies & The Mechanics of Laziness	4
4 Chapter 4: Category Theoretic Abstractions: Functors, Applicatives & Monads . . .	5
II Part II: Solutions for Front-End Architecture & Syntax Analysis	7
1 Chapter 6: Lexical & Syntactic Analysis: Alex & Happy in Practice	7
2 Chapter 7: Abstract Syntax Trees & Expression Evaluation: The Calculator System .	8
III Part III: Solutions for Back-End Construction & Code Generation	10
1 Chapter 8: Stack Machine Architecture: The 17-Instruction Virtual Machine . . .	10
2 Chapter 9: Code Generation: Compiling High-Level Languages to Stack Bytecode .	12
3 Chapter 10: Functions, Activation Records & Runtime Memory Management . . .	13
4 Chapter 11: Advanced Topics: Type Checking, Optimizations & Native Code Generation	14

IV Part IV: Solutions for Course Laboratories & Semester Projects	16
1 Problem Set 1: Lexing and Parsing Well-Parentthesized Expressions	16
2 Problem Set 2 & 3: Architectures for the 24 Calculator and 42 Pascal Milestones .	16

Part I: Solutions for Functional Programming Foundations

Chapter 1: Paradigms of Computation & Lambda Calculus

Exercise 1 (Function Invocation Syntax). *[Ref: intro_haskell.pdf, Slide 5]*

Problem Statement:

How do you invoke the recursive factorial function `fact` with the integer argument `5` in Haskell? Explain the syntactic difference between Haskell's application convention and traditional imperative languages.

Complete Solution & Derivation:

In Haskell, function application is denoted simply by juxtaposition without parentheses:

```
fact 5
```

HASKELL

In imperative languages (e.g. C, Python, Java), function calls require parenthesized tuples: `fact(5)`. In Haskell, writing `fact(5)` is parsed as applying `fact` to the parenthesized grouping `(5)`, which happens to evaluate to the integer `5`. However, for multi-argument functions `f(x, y)`, Haskell treats `(x, y)` as a single 2-tuple argument of type `(a, b)`, which causes a compile-time type mismatch if `f` expects curried arguments `a -> b -> c`.

Exercise 2 (First-Principles Currying & Uncurrying). *[Ref: intro_haskell.pdf, Slide 9]*

Problem Statement:

Implement the foundational functions `curry` and `uncurry` from first principles without using standard library shortcuts:

```
curry    :: ((a, b) -> c) -> a -> b -> c
uncurry  :: (a -> b -> c) -> ((a, b) -> c)
```

HASKELL

Demonstrate that they form an isomorphism by showing that $\text{curry} \circ \text{uncurry} \equiv \text{id}$ and $\text{uncurry} \circ \text{curry} \equiv \text{id}$.

Complete Solution & Derivation:

```
curry :: ((a, b) -> c) -> a -> b -> c
curry f x y = f (x, y)

uncurry :: (a -> b -> c) -> ((a, b) -> c)
uncurry f (x, y) = f x y
```

HASKELL

Proof of Isomorphism:

1. Let $g :: (a, b) \rightarrow c$. Then:

$$(\text{uncurry } (\text{curry } g)) (x, y) = (\text{curry } g) x y = g(x, y) \quad (0.1)$$

Hence $\text{uncurry } (\text{curry } g) = g$, which means $\text{uncurry} \circ \text{curry} = \text{id}$.

2. Let $h :: a \rightarrow b \rightarrow c$. Then:

$$(\text{curry } (\text{uncurry } h)) x y = (\text{uncurry } h) (x, y) = h x y \quad (0.2)$$

Hence $\text{curry } (\text{uncurry } h) = h$, which means $\text{curry} \circ \text{uncurry} = \text{id}$.

This formalizes the categorical bijection between $C^{A \times B}$ and $(C^B)^A$.

Chapter 2: Algebraic Data Types & Recursive Data Modeling

Exercise 3 (Polymorphic Constructor Inference). [Ref: *intro_haskell.pdf*, Slide 8]

Problem Statement:

Consider the polymorphic inductive data type:

```
data MaListe a = Vide | Elem a (MaListe a) deriving (Show)
```

HASKELL

What is the deduced type of the nested expression `Elem Vide Vide` in GHCi? Explain how the Hindley-Milner type inference engine resolves this term.

Complete Solution & Derivation:

In the definition of `Elem`, the constructor signature is:

```
Elem :: a -> MaListe a -> MaListe a
```

HASKELL

Here, the first argument is `Vide`, whose type is `MaListe b` for a fresh type variable b . The second argument is also `Vide`, whose type is `MaListe (MaListe b)`. Substituting $a = \text{MaListe } b$, the overall type of the expression is:

```
Elem Vide Vide :: MaListe (MaListe b)
```

HASKELL

It represents a list of lists of arbitrary elements b , containing exactly one element (the empty list `Vide`).

Exercise 4 (Typeclass Analysis: ``lookup``). [Ref: *intro_haskell.pdf*, Slide 10]

Problem Statement:

Analyze the full type signature of the association list lookup function:

```
lookup :: Eq a => a -> [(a, b)] -> Maybe b
```

HASKELL

Explain:

1. Why the typeclass constraint `Eq a` is mathematically required.
2. Why the return type is `Maybe b` rather than simply `b` or throwing an exception.

Complete Solution & Derivation:

1. `lookup` iterates through the list of pairs `[(a, b)]` and tests whether the search key matches each pair's first component: `if key == k then Just v else ...`. Because equality `(==)` is not defined for arbitrary types (for example, functions `(->)` cannot have decidable equality), the typechecker requires `a` to belong to `Eq`.
2. An associative list may not contain the queried key. Returning `Maybe b` turns `lookup` into a mathematically **total function**: if found, it yields `Just val`; if exhausted without a match, it yields `Nothing`, enforcing safe handling at compile time without null pointer faults.

Exercise 5 (Laboratory TP 0: Binary Trees and Traversals). [Ref: *intro_haskell.pdf*, Slide 18 (TP 0)]

Problem Statement:

1. Define a polymorphic binary tree data type `Tree a` with empty leaves and value-bearing internal nodes.
2. Implement recursive functions computing:
 - The height of the tree (`height :: Tree a -> Int`).
 - The total number of nodes (`nbNodes :: Tree a -> Int`).
 - The number of leaves (`nbLeaves :: Tree a -> Int`).
3. Implement the three canonical depth-first tree traversals returning `[a]`: in-order (infixe), pre-order (préfixe), and post-order (postfixe).

Complete Solution & Derivation:

```
data Tree a
  = Leaf
  | Node a (Tree a) (Tree a)
  deriving (Eq, Show)

-- 1. Tree Metrics:
height :: Tree a -> Int
height Leaf      = 0
height (Node _ l r) = 1 + max (height l) (height r)

nbNodes :: Tree a -> Int
nbNodes Leaf      = 0
nbNodes (Node _ l r) = 1 + nbNodes l + nbNodes r

nbLeaves :: Tree a -> Int
nbLeaves Leaf      = 1
nbLeaves (Node _ l r) = nbLeaves l + nbLeaves r

-- 2. Tree Traversals:
inOrder :: Tree a -> [a]
inOrder Leaf      = []
inOrder (Node x l r) = inOrder l ++ [x] ++ inOrder r

preOrder :: Tree a -> [a]
preOrder Leaf      = []
preOrder (Node x l r) = [x] ++ preOrder l ++ preOrder r

postOrder :: Tree a -> [a]
```

HASKELL

```
postOrder Leaf      = []
postOrder (Node x l r) = postOrder l ++ postOrder r ++ [x]
```

Chapter 3: Evaluation Strategies & The Mechanics of Laziness

Exercise 6 (Infinite Sequence Generation via Laziness). [Ref: *intro_haskell.pdf*, Slide 11]

Problem Statement:

Implement the infinite sequence of natural numbers $[1, 2, 3, 4, \dots]$:

1. Using an explicit recursive generator function.
2. Using Haskell's core lazy list primitives (`iterate` and arithmetic sequences).

Complete Solution & Derivation:

```
-- 1. Explicit recursive generator:
fromN :: Integer -> [Integer]
fromN n = n : fromN (n + 1)

infList1 :: [Integer]
infList1 = fromN 1

-- 2. Standard lazy list primitives:
infList2 :: [Integer]
infList2 = [1..]

infList3 :: [Integer]
infList3 = iterate (+1) 1
```

HASKELL

Operational Mechanics: Due to non-strict evaluation, calling `take 5 infList1` only expands the thunk chain up to five `(:)` constructor applications, leaving the tail unevaluated in memory.

Exercise 7 (Higher-Order Consecutive Element Sums). [Ref: *intro_haskell.pdf*, Slide 12]

Problem Statement:

Write a point-free or lambda expression taking a list $l = [x_0, x_1, x_2, \dots]$ and returning the pairwise consecutive sums $[x_0 + x_1, x_1 + x_2, \dots]$. Analyze the typing and behavior of its constituent higher-order functions.

Complete Solution & Derivation:

```
consecutiveSums :: Num a => [a] -> [a]
consecutiveSums = \l -> zipWith (+) l (tail l)
```

HASKELL

Analysis of Components:

- `tail l` shifts the stream by one index: $[x_1, x_2, x_3, \dots]$.
- `zipWith` has type `(a -> b -> c) -> [a] -> [b] -> [c]`. It consumes two lists simultaneously and pairs elements with `(+)`.

- Termination: `zipWith` terminates as soon as the shorter list terminates, naturally producing a result with length $\text{length}(l) - 1$.

Exercise 8 (Quicksort and Operator Sections). [Ref: *intro_haskell.pdf*, Slide 14]

Problem Statement:

In the canonical Haskell Quicksort implementation:

```
qs :: Ord a => [a] -> [a]
qs []      = []
qs (p:tl) = (qs $ filter (< p) tl) ++ [p] ++ (qs $ filter (>= p) tl)
```

HASKELL

Explain the syntax `(< p)` and `(>= p)`. What are operator sections, and why is parentheses placement crucial?

Complete Solution & Derivation:

In Haskell, an infix binary operator can be partially applied by enclosing it in parentheses with one argument:

- `(< p)` is a right section: it is syntactically equivalent to `\x -> x < p`.
- `(p <)` is a left section: it is syntactically equivalent to `\x -> p < x`.
- If one omits parentheses and writes `filter < p tl`, Haskell's left-associative application parses this as `((filter <) p) tl`, which fails immediately with a type mismatch because `<` is treated as an expression rather than an applied function argument.

Chapter 4: Category Theoretic Abstractions: Functors, Applicatives & Monads

Exercise 9 (Formal Proof of Monad Laws for `Maybe`). [Ref: *intro_haskell.pdf*, Slide 21]

Problem Statement:

For the standard `Maybe` monad instance:

```
return x      = Just x
Nothing >>= _ = Nothing
Just x  >>= f = f x
```

HASKELL

Prove analytically that the 3 categorical monad laws are satisfied for all values:

1. **Left Identity:** `return a >>= f ≡ f a`.
2. **Right Identity:** `m >>= return ≡ m`.
3. **Associativity:** `(m >>= f) >>= g ≡ m >>= (\x -> f x >>= g)`.

Complete Solution & Derivation:

- **Law 1 (Left Identity):**

$$\text{return } a \gg= f = (\text{Just } a) \gg= f = f a \quad (0.3)$$

Holds directly by definition of `(>=)` on `Just`.

- **Law 2 (Right Identity):**
 - Case $m = \text{Nothing}$: `Nothing >>= return = Nothing = m`.

- Case $m = \text{Just } x$: $(\text{Just } x) \gg= \text{return} = \text{return } x = \text{Just } x = m$.

Holds in all cases.

• **Law 3 (Associativity):**

- Case $m = \text{Nothing}$:

$$(\text{Nothing} \gg= f) \gg= g = \text{Nothing} \gg= g = \text{Nothing} \quad (0.4)$$

$$\text{Nothing} \gg= (\lambda x. f(x) \gg= g) = \text{Nothing} \quad (0.5)$$

- Case $m = \text{Just } x$:

$$((\text{Just } x) \gg= f) \gg= g = (f x) \gg= g \quad (0.6)$$

$$(\text{Just } x) \gg= (\lambda x. f(x) \gg= g) = (\lambda x. f(x) \gg= g) x = (f x) \gg= g \quad (0.7)$$

Both expressions evaluate to $(f x) \gg= g$. Hence associativity holds unconditionally.

Exercise 10 (Peeling Monadic Structure (``join``)). [Ref: *intro_haskell.pdf*, Slide 24]

Problem Statement:

The flattening operation (peeling one monadic layer) is defined as:

```
join :: Monad m => m (m a) -> m a
```

HASKELL

1. Implement `join` specifically by hand for `Maybe (Maybe a) -> Maybe a`.
2. Implement `join` universally for any arbitrary `Monad m` using only `(>=)` and `id`.

Complete Solution & Derivation:

```
-- 1. Explicit pattern matching for Maybe:
```

HASKELL

```
joinMaybe :: Maybe (Maybe a) -> Maybe a
joinMaybe Nothing      = Nothing
joinMaybe (Just Nothing) = Nothing
joinMaybe (Just (Just x)) = Just x
```

```
-- 2. Universal categorical definition for any Monad:
```

```
join :: Monad m => m (m a) -> m a
join mma = mma >= id
```

Explanation: In `mma >= id`, `mma` has type `m (m a)` and `id` has type `m a -> m a`. Applying `(>=) :: m t -> (t -> m a) -> m a` with $t = m a$ yields the result type `m a`.

Exercise 11 (List Monad Desugaring & Sequential I/O Printing). [Ref: *intro_haskell.pdf*, Slide 26]

Problem Statement:

1. Desugar the following list comprehension `do`-block into explicit `(>=)` and `return`, verifying the typing at each step:

```
do { x <- [1, 2]; y <- [1, 5]; return (x, y) }
```

HASKELL

2. Implement a function `printAll :: [String] -> IO ()` that prints each string on a separate line using only pure recursion or monadic bind, without using `mapM_` from Prelude.

Complete Solution & Derivation:

1. Desugared List Computation:

```
[1, 2] >>= (\x -> [1, 5] >>= (\y -> return (x, y)))
```

HASKELL

Step-by-step type check:

- Inner lambda: `\y -> return (x, y) :: Int -> [(Int, Int)]`.
- Inner bind: `[1, 5] >>= ...` has type `[(Int, Int)]` (computes pairs for a given x).
- Outer lambda: `\x -> ...` has type `Int -> [(Int, Int)]`.
- Outer bind: `[1, 2] >>= ...` concatenates the sublists: `[(1, 1), (1, 5), (2, 1), (2, 5)]`.

2. Sequential IO Printer:

```
printAll :: [String] -> IO ()
printAll [] = return ()
printAll (s:ss) = putStrLn s >> printAll ss
```

HASKELL

Part II: Solutions for Front-End Architecture & Syntax Analysis

Chapter 6: Lexical & Syntactic Analysis: Alex & Happy in Practice

Exercise 12 (Parenthesized Expression Parsing & Max Depth). [Ref: *compil_happy.pdf*, Section 4.5 (*paren-01*, *paren-02*)]

Problem Statement:

1. Write a Happy grammar over tokens `'('` and `')'` that parses well-parenthesized expressions and returns the maximum nesting depth directly as an integer.
2. Explain how source position tracking via Alex's `%wrapper "posn"` enables accurate error diagnostics.

Complete Solution & Derivation:

```
-- Happy grammar computing nesting depth:
Expr : {- empty -}           { 0 }
      | '(' Expr ')' Expr    { max (1 + $2) $4 }
```

HASKELL

Nesting Depth Deduction:

- Empty string: 0.
- `"()"` : `max(1 + 0, 0) = 1`.
- `"(( ))"` : inner subexpressions have depth 1, outer nesting adds 1, yielding 2.

In Alex, specifying `%wrapper "posn"` wraps each matched token in a record containing `AlexPn offset line col`. When Happy triggers the `%error` routine on unexpected tokens, the compiler extracts `line` and `col` to print precise source location diagnostics.

Exercise 13 (Lexical Identifiers in Context-Free Grammars). [Ref: *compil_happy.pdf*, Section 4.5 (*paren-04*, *paren-05*)]

Problem Statement:

In `paren-04`, the Happy grammar allows identifier tokens only at leaf positions:

```
Expr : {- empty -}      { Empty }
      | ident           { Leaf $1 }
      | '(' Expr ')' Expr { Node $2 $4 }
```

HASKELL

1. Where are identifiers forbidden by this grammar? Give an example of an invalid string.
2. How can the grammar be refactored to allow arbitrary sequences of identifiers and parenthesized groups?

Complete Solution & Derivation:

1. Identifiers cannot appear consecutively (e.g. `"foo bar"`) or immediately after a closed parenthesis without an intervening left parenthesis (e.g. `"(foo) bar"` is parsed as `Node (Leaf "foo") (Leaf "bar")`, but `"(foo) bar baz"` fails because `Expr` on the right expects either empty, a single ident, or a parenthesized block).
2. To permit arbitrary sequences, refactor `Expr` into a list of atomic items:

```
Expr  : ItemList          { $1 }
ItemList : {- empty -}    { [] }
          | Item ItemList { $1 : $2 }
Item   : ident           { IdentItem $1 }
          | '(' Expr ')'  { GroupItem $2 }
```

HASKELL

Chapter 7: Abstract Syntax Trees & Expression Evaluation: The Calculator System

Exercise 14 (Pure Arithmetic AST & Evaluator). [Ref: *compil_happy.pdf*, Section 3.1–3.2]

Problem Statement:

1. Define a pure algebraic data type `AExpr` supporting floating-point literals (`Double`), unary negation, and the binary operators `+`, `-`, `*`, `/`, and `^`.
2. Implement an evaluator `eval :: AExpr -> Maybe Double` that safely returns `Nothing` when encountering a division by zero.

Complete Solution & Derivation:

```
data AExpr
  = Lit Double
  | Neg AExpr
  | Add AExpr AExpr
```

HASKELL

```

    | Sub AExpr AExpr
    | Mul AExpr AExpr
    | Div AExpr AExpr
    | Pow AExpr AExpr
    deriving (Eq, Show)

eval :: AExpr -> Maybe Double
eval (Lit d)      = Just d
eval (Neg e)      = fmap negate (eval e)
eval (Add e1 e2)  = do { v1 <- eval e1; v2 <- eval e2; return (v1 + v2) }
eval (Sub e1 e2)  = do { v1 <- eval e1; v2 <- eval e2; return (v1 - v2) }
eval (Mul e1 e2)  = do { v1 <- eval e1; v2 <- eval e2; return (v1 * v2) }
eval (Pow e1 e2)  = do { v1 <- eval e1; v2 <- eval e2; return (v1 ** v2) }
eval (Div e1 e2)  = do
    v1 <- eval e1
    v2 <- eval e2
    if v2 == 0 then Nothing else Just (v1 / v2)

```

Exercise 15 (Environment-Based Evaluation & Variables). [Ref: *compil_happy.pdf*, Section 3.4 & Section 5]

Problem Statement:

Extend `AExpr` with variable names `Var String` and assignments `Assign String AExpr`. Formulate an evaluator `evalEnv :: [(String, Double)] -> AExpr -> Either String (Double, [(String, Double)])` that looks up bound variables, rejects unbound identifiers with a descriptive error message, and updates the environment upon assignment.

Complete Solution & Derivation:

```

type Env = [(String, Double)]

evalEnv :: Env -> AExpr -> Either String (Double, Env)
evalEnv env (Lit d) = Right (d, env)
evalEnv env (Var x) = case lookup x env of
    Just v  -> Right (v, env)
    Nothing -> Left ("Undefined variable: " ++ x)
evalEnv env (Assign x e) = do
    (val, env') <- evalEnv env e
    let newEnv = (x, val) : filter ((/= x) . fst) env'
    return (val, newEnv)
evalEnv env (Add e1 e2) = do
    (v1, env1) <- evalEnv env e1
    (v2, env2) <- evalEnv env1 e2
    return (v1 + v2, env2)
evalEnv env (Div e1 e2) = do
    (v1, env1) <- evalEnv env e1
    (v2, env2) <- evalEnv env1 e2
    if v2 == 0
    then Left "Evaluation error: division by zero"
    else return (v1 / v2, env2)

```

HASKELL

Part III: Solutions for Back-End Construction & Code Generation

Chapter 8: Stack Machine Architecture: The 17-Instruction Virtual Machine

Exercise 16 (Relational Comparison on the 17-Instruction Machine). [Ref: *defmachine.pdf*, Slide 9]

Problem Statement:

The virtual machine does not possess a primitive comparison instruction (such as `<` or `CMP`). It provides only `BEZ` (branch if zero) and `BGZ` (branch if strictly positive). Given two values A and B at the top of the stack (with B above A), write a minimal symbolic assembly sequence computing the boolean test $A < B$, leaving 1 on the stack if true and 0 if false.

Complete Solution & Derivation:

To determine if $A < B$, compute the difference $B - A$:

$$A < B \Leftrightarrow B - A > 0 \quad (0.8)$$

Assembly Sequence:

```

; Stack initially contains: [..., A, B]
SWAP      ; Stack: [..., B, A]
SUB       ; Stack: [..., B - A]
BGZ is_true ; Pops (B - A); jumps if (B - A) > 0
PUSH 0    ; False case: push 0
PUSH end_cmp
GOTO
is_true EQU *
PUSH 1    ; True case: push 1
end_cmp EQU *
```

ASSEMBLY

After this sequence, the condition value (0 or 1) is at the top of the stack.

Exercise 17 (Manual Assembly Compilation: Euclidean GCD). [Ref: *happy_pascal.pdf*, Milestone 14 & *defmachine.pdf*]

Problem Statement:

Write the complete symbolic assembly program calculating the Greatest Common Divisor (gcd) of two integers read from standard input, using the Euclidean algorithm, and printing the result to `OUT`.

Complete Solution & Derivation:

```

; Variable data allocations
x DS 1
y DS 1
r DS 1
```

ASSEMBLY

```
;/ Read inputs into variables x and y
PUSH x
IN
STORE
PUSH y
IN
STORE

gcd_loop EQU *
;/ Condition: while y != 0
PUSH y
LOAD
BEZ gcd_end

;/ Body: r := x mod y = x - (x / y) * y
PUSH r
PUSH x
LOAD
PUSH x
LOAD
PUSH y
LOAD
DIV
PUSH y
LOAD
MUL
SUB
STORE      ;/ r = x - (x / y) * y

;/ x := y
PUSH x
PUSH y
LOAD
STORE

;/ y := r
PUSH y
PUSH r
LOAD
STORE

;/ Repeat loop
PUSH gcd_loop
GOTO

gcd_end EQU *
;/ Print x to standard output
PUSH x
LOAD
OUT
STOP
```

Chapter 9: Code Generation: Compiling High-Level Languages to Stack Bytecode

Exercise 18 (Recursive AST to Stack Bytecode Emission). [Ref: *happy_pascal.pdf*, Milestones 1–6]

Problem Statement:

Consider an arithmetic expression AST type:

```
data Op = Add | Sub | Mul | Div
data Expr = Lit Int | Var String | BinOp Op Expr Expr
```

HASKELL

Write a pure code generation function `compileExpr :: Expr -> [String]` that emits assembly instructions maintaining the stack invariant (leaving exactly one result at the top of the operand stack).

Complete Solution & Derivation:

```
compileExpr :: Expr -> [String]
compileExpr (Lit n)    = ["PUSH " ++ show n]
compileExpr (Var x)    = ["PUSH " ++ x, "LOAD"]
compileExpr (BinOp op e1 e2) =
  compileExpr e1 ++
  compileExpr e2 ++
  [emitOp op]
  where
    emitOp Add = "ADD"
    emitOp Sub = "SUB"
    emitOp Mul = "MUL"
    emitOp Div = "DIV"
```

HASKELL

Operational Invariant: Evaluating `e1` pushes its value; evaluating `e2` pushes its value. Then the binary operation pops both values and pushes the combined result. The stack remains strictly balanced.

Exercise 19 (Array Indexing L-Value & R-Value Emission). [Ref: *happy_pascal.pdf*, Milestone 15 & *defmachine.pdf*]

Problem Statement:

1. Specify the stack assembly sequence to read the value of `primes[i + 1]`.
2. Specify the stack assembly sequence to execute the assignment `primes[i + 1] := 42;`.

Complete Solution & Derivation:

1. Reading `primes[i + 1]` (R-Value):

```
PUSH primes ;/ Base address
PUSH i      ;/ Evaluate index expression (i + 1)
LOAD
PUSH 1
ADD
```

ASSEMBLY

```
ADD      ;/ Effective address = primes + (i + 1)
LOAD     ;/ Read memory content onto stack top
```

2. Writing `primes[i + 1] := 42` (L-Value assignment):

```
;/ 1. Compute destination address (L-value)
PUSH primes
PUSH i
LOAD
PUSH 1
ADD
ADD      ;/ Target address is now on stack
;/ 2. Evaluate right-hand expression
PUSH 42
;/ 3. Store into memory
STORE    ;/ data[target_addr] = 42; sp -= 2
```

ASSEMBLY

Chapter 10: Functions, Activation Records & Runtime Memory Management

Exercise 20 (Recursive Function Activation & Factorial Emission). [Ref: *happy_pascal.pdf*, Milestone 28 & *intro_haskell.pdf*, Slide 5]

Problem Statement:

Outline the stack machine calling convention for the recursive function:

```
function fact(n: integer): integer;
begin
  if n <= 1 then fact := 1 else fact := n * fact(n - 1)
end;
```

PASCAL

Provide the symbolic assembly implementation with return address preservation and frame teardown.

Complete Solution & Derivation:

```
;/ Calling sequence from main:
PUSH ret_main
PUSH 5      ;/ Pass argument n = 5
PUSH fact_entry
GOTO
ret_main EQU *
OUT      ;/ Prints 120
STOP

fact_entry EQU *
;/ Stack top: [return_addr, n]
SWAP     ;/ Stack: [n, return_addr]
;/ Test base case: n <= 1
```

ASSEMBLY

```

; Compute difference (1 - n)
PUSH 1
PUSH n_local
STORE          ; Store n into temporary or frame slot
...
; In a pure stack approach without frame registers:
; Save return address, compute recursive call, multiply, and exit:
SWAP
GOTO          ; Indirect jump back to saved return address

```

Exercise 21 (Fibonacci: Naive Tree Recursion vs. Linear Accumulator). [Ref: *happy_pascal.pdf*, Milestone 28]

Problem Statement:

1. Write the naive recursive Fibonacci function $F(n) = F(n-1) + F(n-2)$ and analyze its stack frame growth.
2. Implement an optimized linear tail-recursive accumulator version in Haskell and explain why it compiles to $\mathcal{O}(1)$ stack memory.

Complete Solution & Derivation:

```

-- 1. Naive recursive Fibonacci (O(2^n) time, O(n) call stack depth):
fibNaive :: Integer -> Integer
fibNaive 0 = 0
fibNaive 1 = 1
fibNaive n = fibNaive (n - 1) + fibNaive (n - 2)

-- 2. Tail-recursive linear accumulator (O(n) time, O(1) stack space):
fibOptimized :: Integer -> Integer
fibOptimized n = go n 0 1
  where
    go 0 !a !b = a
    go k !a !b = go (k - 1) b (a + b)

```

HASKELL

Compiler Mechanics: Under Tail Call Optimization (TCO), `go (k - 1) b (a + b)` occupies the tail position. Rather than pushing an activation record, the compiler updates parameters $k \rightarrow k-1$, $a \rightarrow b$, and $b \rightarrow a+b$ in place and loops back, executing in constant stack memory.

Chapter 11: Advanced Topics: Type Checking, Optimizations & Native Code Generation

Exercise 22 (Constant Folding AST Optimization). [Ref: *happy_pascal.pdf*, Milestone 41]

Problem Statement:

Implement a recursive optimization pass `foldConstants :: Expr -> Expr` on arithmetic expressions that collapses subtrees containing only constant integer literals into single literal nodes at compile time.

Complete Solution & Derivation:

```

data Op = Add | Sub | Mul | Div deriving (Eq, Show)
data Expr
  = Lit Int
  | Var String
  | BinOp Op Expr Expr
  deriving (Eq, Show)

foldConstants :: Expr -> Expr
foldConstants (BinOp op e1 e2) =
  case (foldConstants e1, foldConstants e2) of
    (Lit c1, Lit c2) -> Lit (evalConstant op c1 c2)
    (e1', e2')      -> BinOp op e1' e2'
foldConstants leaf = leaf

evalConstant :: Op -> Int -> Int -> Int
evalConstant Add = (+)
evalConstant Sub = (-)
evalConstant Mul = (*)
evalConstant Div = div

```

HASKELL

Optimization Benefit: Subterms like `(2 + 3) * x` are simplified to `5 * x` before generating bytecode, eliminating four virtual stack instructions at run time.

Exercise 23 (Short-Circuit Boolean Evaluation). *[Ref: happy_pascal.pdf, Milestone 38]*

Problem Statement:

Pascal's logical operators `and` and `or` can cause unwanted side effects or division-by-zero crashes if both operands are eagerly evaluated (e.g. `(y <> 0)` and `(x mod y == 0)`). Formulate the conditional jump pattern that guarantees short-circuit execution on the stack machine.

Complete Solution & Derivation:

1. Compiling `e1 and e2` (Short-Circuit):

```

;/ Evaluate e1
<code for e1>
BEZ short_false ;/ If e1 == 0, false! Skip e2
<code for e2>
BEZ short_false ;/ If e2 == 0, false!
PUSH 1          ;/ Both were non-zero: true
PUSH end_and
GOTO
short_false EQU *
PUSH 0
end_and EQU *

```

ASSEMBLY

Guarantee: If `e1` evaluates to 0, the machine jumps directly to `short_false`, and the instructions for `e2` are never executed.

Part IV: Solutions for Course Laboratories & Semester Projects

Problem Set 1: Lexing and Parsing Well-Parentthesized Expressions

Exercise 24 (Parenthesis Depth Parser). [Ref: *compil_happy.pdf* (paren-01)]

Problem Statement:

Write a Happy grammar that directly computes the maximum nesting depth of well-parentthesized expressions as an integer.

Complete Solution & Derivation:

```
Expression : {- empty -}                { 0 }
            | '(' Expression ')' Expression { max (1 + $2) $4 }
```

HASKELL

Verification:

- For "" : matches empty rule $\rightarrow 0$.
- For "()" : $1 + 0 = 1$, followed by $0 \rightarrow \max(1, 0) = 1$.
- For "()" : inner is 1, outer is $1 + 1 = 2$.

Exercise 25 (Rose Tree Parse Generation). [Ref: *compil_happy.pdf* (paren-03)]

Problem Statement:

Modify the grammar to produce an explicit multi-way tree (generalized rose tree) representing the nested syntactic hierarchy.

Complete Solution & Derivation:

```
data Parens = Node [Parens] deriving (Eq, Show)

Expr : {- empty -}                { [] }
      | '(' Expr ')' Expr { Node $2 : $4 }
```

HASKELL

Problem Set 2 & 3: Architectures for the 24 Calculator and 42 Pascal Milestones

Exercise 26 (Incremental Calculator Pipeline Architecture (24 Milestones)). [Ref: *compil_happy.pdf*, Section 5]

Problem Statement:

How should the scanner (*CalculatriceLexer.x*), parser (*CalculatriceParser.y*), and runtime REPL (*Calcul.hs*) be structured to systematically satisfy all 24 milestones (from Double numbers to Unicode variables and implicit multiplication)?

Complete Solution & Derivation:

1. **Lexer (`CalculatriceLexer.x`):** Use `%wrapper "posn"` . Match numbers with scientific exponents `$digit+(\. $digit+)?([eE][\+|-]?$digit+)?` , keywords `true` , `false` , `quit` , `exit` , identifier strings `$alpha[$alpha $digit]*` , Unicode `π` , and discard whitespace `$white+` ; and comments `//.* ;` .
2. **Parser Precedence (`CalculatriceParser.y`):**

```
%right '='
%left '?' ':'
%left "or"
%left "and"
%nonassoc "==" "!=" '<' "<=" '>' ">="
%left '+' '-'
%left '*' '/'
%right '^'
%left NEG
```

HASKELL

3. **Stateful REPL (`Calcul.hs`):** The loop carries a state environment type `Env = [(String, Double)]` initialized with `[("pi", pi), ("e", exp 1), ("π", pi)]` . Each evaluated assignment updates the map and recursively passes the new environment to the next loop iteration.

Exercise 27 (Pascal Compiler Code Generator Architecture (42 Milestones)). [Ref:*happy_pascal.pdf, Section 2]***Problem Statement:**

Provide the overall architecture for the 42-stage Pascal compiler translating Pascal ASTs into 17-instruction symbolic bytecode, specifically addressing label generation, memory reservation, and activation records.

Complete Solution & Derivation:

1. **Label Generation Monad:** Code generation uses a `State Int` monad or pure counter `freshLabel :: String -> State Int String` producing monotonic symbolic labels (`L_else_1` , `L_end_1` , etc.).
2. **Data Allocation Pass:** Before code emission, an AST pass extracts global `var` declarations and emits memory allocation directives:

```
x    DS 1
arr  DS 100
```

ASSEMBLY

3. **Stack Calling Convention:** For functions (Milestones 22–28), the caller pushes actual arguments and the return label, and branches via `GOTO` . The callee swaps the computed return value onto the stack, retrieves the return label, and branches back via `SWAP \n GOTO` .